④ Entry & exit :- Ticket based. RFID.

⑤ Floors :- single / multiple floors

⑥ How many entrance / exit :- Now 1 → extensible more

ext:
→ Floors. Nearest Parking find.

---

classes & objects

① vehicle :- → no
→ type → 2 wheel
4 wheel

② Ticket :- Entry Time
Parking spot Id.
vehicle type

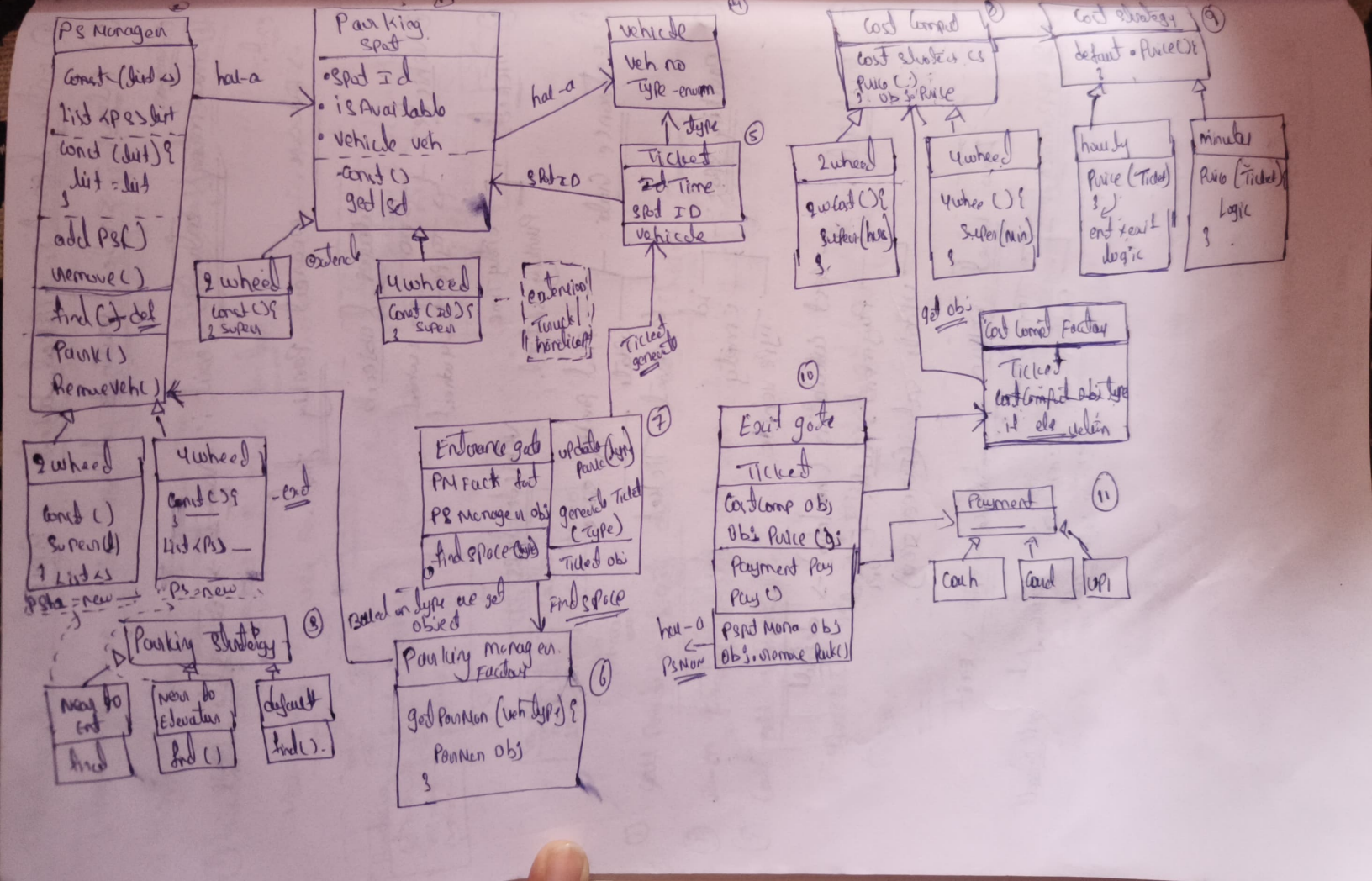
③ Entrance Gate :- Find Parking spot
update
Generate Ticket

④ parking spot :- id
Empty
Type vehicle

⑤ Exit gate :- cost calculation (hrs/min) → default
Payment. Cash. Card / UPI
update spot. (Free spot).

Low Level Design

→ 2 Approaches → Top-Down → Entrance → spot → Exit
Bottom-up → Implement basic 1st. all small
Integrate all.


PS Manager
Parking Spot
Vehicle
Cost Computation
Cost Strategy
Ticket
2 wheel
4 wheel
hourly
minutes
Entrance gate
Exit gate
Parking Strategy
Parking Manager Factory
Cost Computation Factory
Payment
Cash
Card
UPI
has-a
extends
get obj

① Park Spot :- we need this to get isAvailable() → Space.

→ 2 Types < 2 wheel / 4 wheel → Both extends.

→ we need type of vehicle to send availability - how-a vehicle

② Manager :- In single floor, there will be 100 of Spots.

→ This List maintained by Manager.

→ get List based on 2 wheel & 4 wheel.

→ This has all method - ① find ② Add Remove ③ Park ④ unPark

③ ~~Factory~~ Strategy :- Manager gives result based on vehicle type

→ This type will be sent by Strategy & gets object

→ And we call methods

④ Vehicle / Ticket → Provides the basic info which they Contains

⑤ Entrance gate → calls PS ~~Manager~~ Parking & provide vehicle type

→ ~~manager~~ gets Facility gets manager obj < 2 / 4.

→ find the space available & get Id of spot

⑥ Cost computation → used by exit gate → to generate price.

2 types < 2 / 4. Logic will be there.

⑦ Cost Strategy :- < hour / minute → here there will be diff logic

⑧ Factory :- gets object & return to exit gate.

⑨ Exit gate :- get obj from factory & call obj.Price(2 wheeler) has

→ calls Payment method. again types

→ Call Park spot Manager & get object & free up the space.